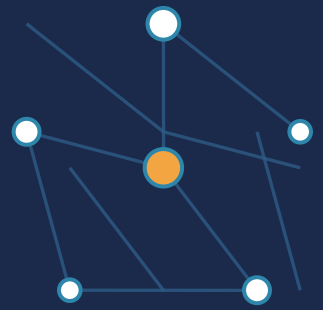
COURS — WEB SÉMANTIQUE

Introduction au Web Sémantique

2ème année Mastère Professionnel en Système d'Information Décisionnel

Enseignante : Safa Chmengui



PLAN DU COURS

Sommaire

01 **Du Web au XML**

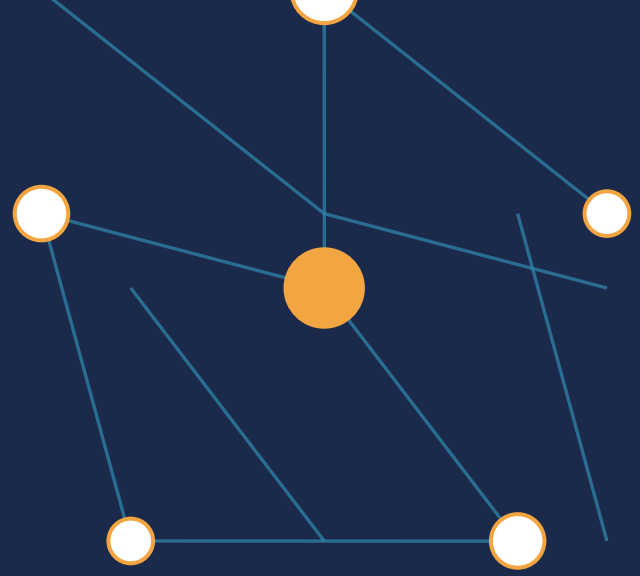
Historique, HTML, SGML, XML, et les technologies satellites (DTD, XSL, DOM, SAX).

02 **Les limites du Web actuel**

Pourquoi un Web « syntaxique » ne suffit plus aux machines : vers le Web sémantique.

03 **Cours Partie II**

RDF, RDF/XML, RDFS en détail, avec les URI, l'exemple Dublin Core, le graphe RDFS et OWL.



PARTIE 1

Du Web au XML

De l'hypertexte aux langages de balisage structurés



Historique : du Web 1.0 au Web 3.0

Web 1.0

Web statique

Pages HTML (texte, liens, images) publiées par les entreprises, consultées via http. Sites rarement mis à jour.

Repères
1969 · ARPANET
1990 · premier navigateur Web
1994 · création du W3C

Web 1.5

Web dynamique

Scripts (PHP, JavaScript) + bases de données : pages dynamiques, apparition des forums.

Web 2.0

Web participatif

Les internautes produisent le contenu : « consomm'acteur ». Phénomène plus sociologique que technique.

Web 3.0

Web sémantique

Les connaissances sont formalisées pour être interprétées par les machines (métadonnées).

Le Web 2.0 correspond au Web tel qu'imaginé par Tim Berners-Lee : chacun peut publier et consommer de l'information.

...HTML

- Le Web traditionnel repose sur l'hypertexte : on a donc écrit le langage HTML (HyperText Markup Language).
- HTML a été conçu comme un langage spécifiant le contenu d'un document, avec d'importantes extensions d'hypertexte.
- Il n'est pas conçu pour être un langage de traitement de texte WYSIWYG (comme Word) : un même document HTML peut être affiché par des navigateurs très différents.

HTML… notions et règles

- Langage de base du Web : un langage à balises permettant la présentation de documents et des liens simples entre documents.
- Les commandes HTML ont une marque de début et une marque de fin (certaines marques de fin sont facultatives).
- Les commandes HTML utilisent les caractères < et > comme délimiteurs.
- Exemple : <TITLE>Cette commande donne un titre au document HTML</TITLE>
- Les commandes HTML peuvent être écrites en minuscules ou en majuscules.
- Un fichier rédigé en HTML doit être suffixé par .html ou .htm.

HTML… les en-têtes

`<HTML> </HTML>`	Commandes de début et de fin du document HTML.
`<HEAD> </HEAD>`	Informations non affichées concernant le document.
`<TITLE> </TITLE>`	Titre du document — apparaît dans la barre de titre du client WWW.
`<META ...>`	Situées dans l'en-tête, ces commandes guident les moteurs de recherche pour indexer la page.
`<BODY> </BODY>`	Corps du document — toutes les informations affichées y sont contenues.

Pour insérer des commentaires : <!-- Commentaires -->

HTML... exemple

```
<HTML>
  <HEAD>
    <TITLE>Exemple de structure de document HTML</TITLE>
    <META NAME="Author" CONTENT="Cours HTML">
    <BASE HREF="/usr/home/chezmoi"></BASE>
  </HEAD>
  <BODY> ... Le document HTML ... </BODY>
</HTML>
```

Limites HTML…

HTML laisse l'interprétation des éléments marqués au navigateur, ce qui donne une interprétation différente suivant le navigateur utilisé. Mais chacun y a ajouté des extensions propriétaires (ex. les « frames » de Netscape), si bien qu'on est arrivé à une foison de balises spécifiant l'apparence — au point de noyer le contenu !

...SGML

- Face à ce problème de séparation contenu/forme, on a développé un langage structuré : SGML (Standard Generalized Markup Language).

- SGML a été conçu comme un langage spécifiant le contenu d'un document, avec d'importantes extensions d'hypertexte — mais sans en spécifier la forme à l'origine.

SGML… notions

- Issu d'un besoin élémentaire : rendre le stockage de données indépendant de tout fournisseur de logiciels.

- Pas de balises prédéfinies comme dans HTML : le développeur peut créer ses propres balises.

- La DTD (Document Type Definition) est au cœur d'une application SGML :

  - métadonnée définissant l'imbrication des éléments d'un document (obligatoires, optionnels, répétitifs…), avec deux types de données de base (#PCDATA).

- Introduit la notion de document valide (conforme à la DTD), contrairement à HTML qui ne fait que vérifier la syntaxe des balises (document « bien formé »).

- Beaucoup trop de fonctions (méta-langage permettant par exemple de définir les caractères de balisage) avec une syntaxe complexe.

Limite SGML… → XML

SGML a été très utilisé pour le traitement de la documentation technique dans l'industrie (aviation, automobile, etc). Mais c'est un langage complexe — on a donc développé un nouveau langage : XML.

L'idée de départ de XML était de créer un langage ayant 80% de l'expressivité de SGML et 20% de sa complexité.

XML… notions

- XML (eXtensible Markup Language) permet de générer des langages de balises pour structurer données et documents — il dérive de SGML, mais avec des règles moins strictes.
- Du HTML, XML hérite l'utilisation des adresses Web (URL) pour pointer vers d'autres objets.
- De SGML, il hérite des mécanismes d'adressage puissants, permettant par exemple de pointer vers des parties ou des plages de documents.
- XML est fondé sur Unicode, qui gère aussi bien les caractères accentués que les caractères asiatiques — contrairement à HTML et SGML, fortement liés à l'ASCII. C'est la clé de l'internationalisation.

XML… exemple

```
<catalogue>
  <roller>
    <marque>RollerBlade</marque>
    <modele>Coyote</modele>
  </roller>
  <roller>
    <marque>Mission</marque>
    <modele>RL
      <type>
        <roulement>ABEC3608</roulement>
      </type>
    </modele>
  </roller>
</catalogue>
```

DTD… exemple

```
<!DOCTYPE biblio[
  <!ELEMENT biblio (livre)*>
  <!ELEMENT livre (titre, auteur, nb_pages)>
  <!ATTLIST livre
    type (roman | nouvelles | poemes | theatre) #IMPLIED
    lang CDATA "fr"
  >
  <!ELEMENT titre (#PCDATA)>
  <!ELEMENT auteur (#PCDATA)>
  <!ELEMENT nb_pages (#PCDATA)>
]>
```

XML… avantages

- Structure logique de type arborescent, semblable à HTML mais dénuée de toute information de présentation.

- Représenté sous forme de fichier texte : un simple éditeur de texte permet son débogage.

- Format d'échange très pratique entre plates-formes différentes : format de stockage générique, déclinable dans des formats et types d'édition différents, et format d'échange à part entière.

XML… limites

Le fichier créé est « bavard ». Cependant, l'espace disque étant de moins en moins coûteux et les méthodes de compression de plus en plus efficaces, cela ne représente pas un vrai problème : la répétition d'un tag (balise ouvrante / balise fermante) est justement le genre de redondance qui disparaît bien à la compression.

Autour d'XML

Il existe un grand nombre d'autres recommandations et extensions autour d'XML :

DTD

Document Type Definition — décrit la grammaire d'un document XML (voir slide précédente).

XPath

Un langage (non XML) pour localiser une portion d'un document XML.

XPointer

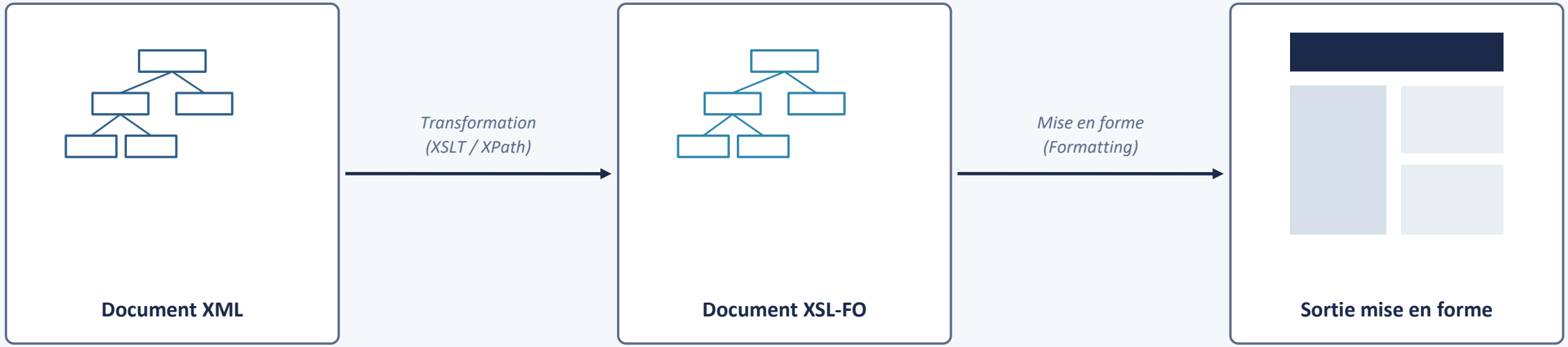
Spécification du W3C permettant de désigner un fragment de document XML en ligne. Utilise la syntaxe XPath, enrichie d'options pour désigner des portions de document (range).

XLink

Spécification du W3C permettant de créer des liens entre fichiers XML ou portions de fichiers (via XPointer). Contrairement aux liens HTML, XLink permet de relier plus de deux fichiers.

Autour d'XML — XSL

XSL (eXtensible Stylesheet Language) est le langage de description de feuilles de style du W3C associé à XML. Sa spécification est divisée en trois parties : XSLT (transformation), XPath (navigation) et XSL-FO (mise en forme).



Autour d'XML — DOM & SAX

DOM

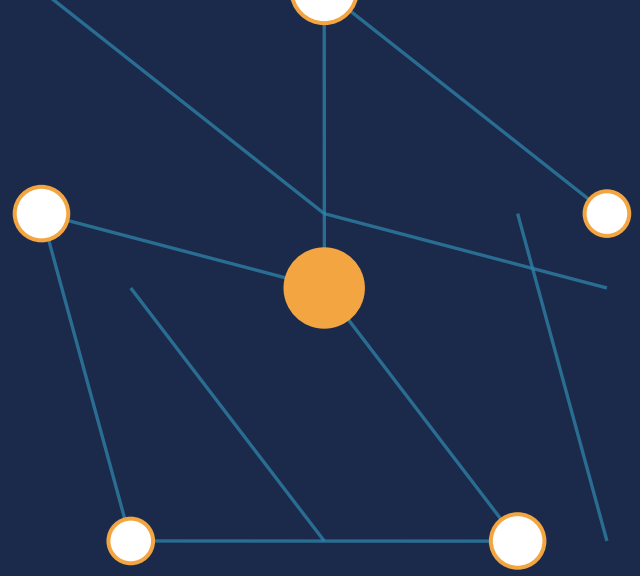
Document Object Model

Constitue un objet en mémoire de la totalité d'un document XML. Cette API permet l'accès direct à tous les nœuds de l'arbre (éléments, texte, attributs), pour les lire ou les modifier.

SAX

Simple API for XML

Alternative intéressante à DOM pour le traitement de documents longs : capture des événements (ouverture/fermeture de balise) au fil de la lecture, sans charger tout le document en mémoire.



PARTIE 2

Les limites du Web actuel

Pourquoi la structure syntaxique ne suffit pas aux machines



Un Web syntaxique…

Le Web actuel est syntaxique : la structure des ressources est bien définie, mais le contenu reste quasi inaccessible aux machines. Seuls les humains peuvent l'interpréter.

Recherche textuelle : les limites

Recherche d'informations sur un livre dont le sujet est Kateb Yacine.

1 **Recherche textuelle** le moteur de recherche ne s'intéresse, par exemple, qu'aux termes « livre » et « Kateb Yacine ».

2 **Résultat obtenu** le moteur retournera les livres écrits PAR cet auteur — mais pas nécessairement les livres qui PARLENT de lui, ni ne distinguera son rôle exact.

L'ambiguïté du vocabulaire

```
<musique genre="classique">
  <titre>Eine Kleine Nacht Music</titre>
  <compositeur>Mozart</compositeur>
</musique>
```

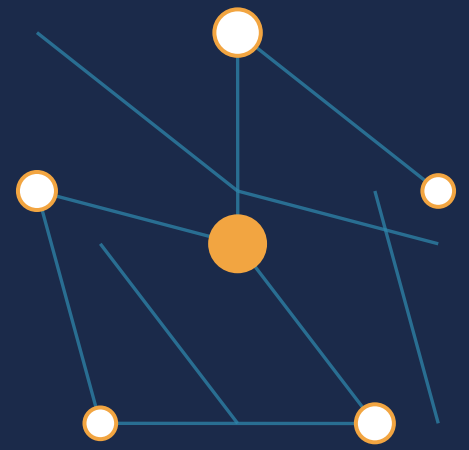
```
<musique genre="classique">
  <titre>Un autre morceau de Mozart</titre>
  <auteur>Mozart</auteur>
</musique>
```

Deux documents décrivent le même type d'information musicale — mais avec des balises différentes (« compositeur » vs « auteur »).

Que se passe-t-il si l'on veut récupérer l'ensemble des morceaux écrits par Mozart ? **Il faut se mettre d'accord sur le sens des mots !**

Vers un Web « intelligent »

Le Web actuel doit changer d'orientation : d'un Web présentable, qui se contente d'afficher les informations disponibles, à un Web « intelligent », permettant aux machines de mieux exploiter les informations !



« The Semantic Web is an extension of the current web in which information is given well defined meaning, better enabling computers and people to work in cooperation. »

— Tim Berners-Lee

COURS · PARTIE II

Web Sémantique

RDF, syntaxe RDF/XML et RDF Schema



2ème année Mastère Professionnel en Ingénierie des Systèmes d'Informations Décisionnels

Enseignante : Safa Chmengui

Plan du cours

1 **Le Web sémantique**
Définition, objectifs, principes

2 **Architecture en couches**
URI, XML, RDF, ontologie, logique, preuve, confiance

3 **RDF**
Modèle de triplets, graphe, syntaxe RDF/XML, conteneurs

4 **RDF Schema (RDFS)**
Classes, sous-classes, propriétés, domaine et co-domaine

Le problème : un Web conçu pour les humains

- **Succès du Web** : simplicité, un simple navigateur suffit, peu de connaissances informatiques requises.
- **HTML = langage de présentation** : il décrit l'affichage, pas le sens des données.
- **Contenu non interprétable par les machines** : informations et services « cachés » dans le code HTML.
- **Le Web actuel limite** la recherche, l'extraction, la maintenance et la génération d'information.

Ce que voit l'humain

Les Misérables
Victor Hugo, 1862

Ce que voit la machine

```
<h1>Les Misérables</h1>
<p>Victor Hugo, 1862</p>
→ du texte, sans sens
```

L'idée du Web sémantique : pallier ces limites en rendant le sens explicite.

Web sémantique : définition

> *« Le Web sémantique n'est pas un Web distinct mais un prolongement du Web que l'on connaît, dans lequel on attribue à l'information une signification clairement définie, ce qui permet aux ordinateurs et aux humains de travailler en plus étroite collaboration. »*

Extension du Web

Connaissances formalisées à l'aide de langages interprétables par les machines, et non plus en langue naturelle.

Sens explicite

Rendre explicite le contenu sémantique des ressources (documents, pages, services) via des métadonnées.

Humains + machines

Un environnement où humains et agents logiciels communiquent sur une base sémantique commune.

Objectifs et principes

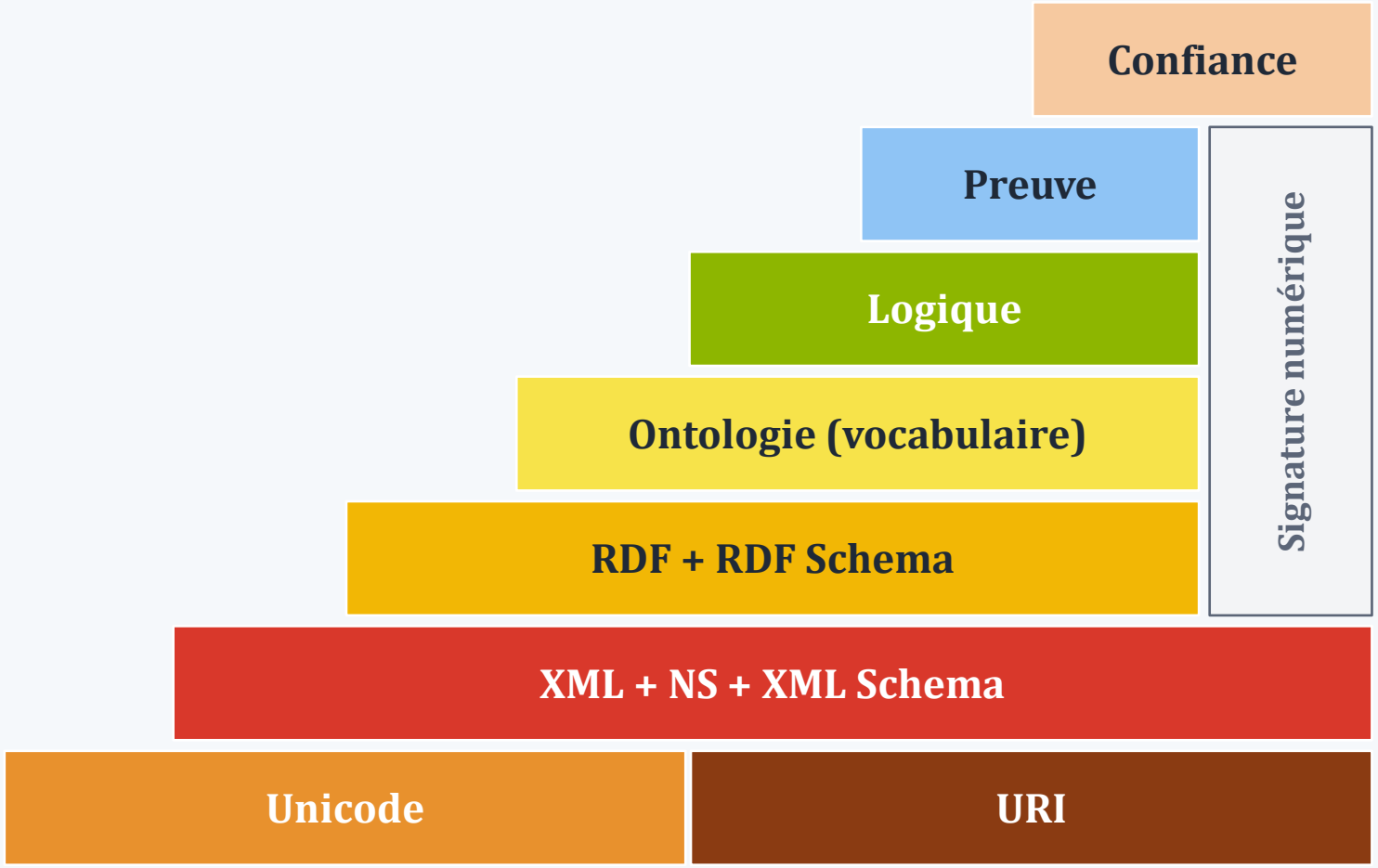
Objectifs

- Infrastructure permettant à des agents logiciels d'aider les utilisateurs dans l'accès aux ressources
- Étendre le réseau d'hyperliens entre pages par un réseau de liens entre données structurées
- Exploiter tout le potentiel du Web
- Laisser les machines réaliser seules les tâches fastidieuses : recherche, association d'informations, action sur le Web

Principes

- Un ensemble de technologies rendant le contenu du Web exploitable par les programmes et agents logiciels
- Les agents peuvent rechercher, filtrer et afficher l'information correspondant au besoin de l'utilisateur
- Ajout d'une couche de modélisation des données → un Web des données :
  - les données sont sur le Web
  - les machines y ont accès
  - des technologies pour l'intégration des données

Architecture du Web sémantique en couches



- **URI + Unicode** : identifier et coder les données
- **XML (+ espaces de noms, XML Schema)** : structurer
- **RDF + RDF Schema** : décrire le sens des données
- **Ontologie** : vocabulaire commun
- **Logique** : règles, déduction de nouveaux faits
- **Preuve** : valider les inférences
- **Confiance** : crédibilité des résultats
- **Signature numérique** : sécurise les couches RDF à preuve

Les couches, une par une (1/2)

Couche URI

Les données sont codées par le standard Unicode et possèdent une adresse URI (Uniform Resource Identifier).

Couche XML

Pour bien structurer les données, on utilise un langage de balises : XML (eXtensible Markup Language), avec espaces de noms (NS) et XML Schema.

Couche RDF

RDF + RDF Schema (Resource Description Framework) : organiser plus finement l'information du Web par une description sémantique des données.

Couche ontologie

Définit une sémantique et un vocabulaire communs aux données du Web, ce qui évite les conflits de partage et d'intégration.

Les couches, une par une (2/2)

Couche logique

Langages d'expression de règles. Ils permettent de déduire de nouveaux faits à partir des faits existants.

Couche preuve

Fournit les moyens de démontrer la validité des inférences produites par les agents.

Couche confiance

Assure la crédibilité des résultats des agents : cryptographie des messages, signatures électroniques.

Dans ce cours, on étudie les couches 2 à 4 : XML (rappel), RDF puis RDF Schema (ontologies légères).

01

RDF

Resource Description Framework : décrire des ressources par des triplets

RDF : qu'est-ce que c'est ?

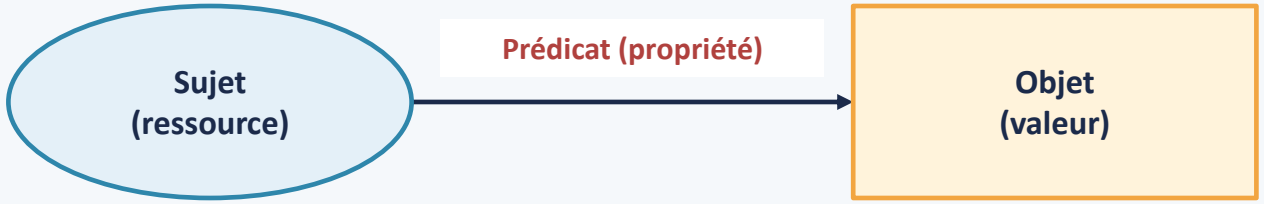
- **Pilier de l'activité Web sémantique du W3C** pour créer des métadonnées (norme : w3.org/RDF).
- **Modèle de données** conceptuel pour décrire des ressources sur le Web.
- **Ressource** : toute entité que l'on veut décrire, même non accessible sur le Web (une personne, une photo, une page, une classe…).
- **Langage de métadonnées simple** pour décrire les ressources et les relations entre elles.
- **Interopérabilité** entre applications qui partagent des informations interprétables par les machines.
- **Syntaxe XML** (mais ce n'est pas la seule).

Objectifs de RDF

- Établir des catalogues de ressources (contenu et relations)
- Spécifier la sémantique des données d'un document XML
- Améliorer la découverte de ressources sur le Web
- Développer des agents intelligents

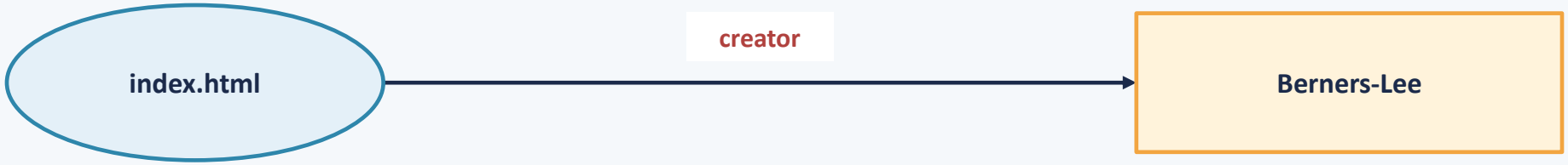
Le modèle RDF : la déclaration (triplet)

- Un modèle RDF est un ensemble de déclarations (statements).
- Une déclaration = un triplet de 3 éléments :
  - **Sujet** : la ressource décrite
  - **Prédicat** : la propriété
  - **Objet** : la valeur



Exemple

index.html a un créateur (creator) dont la valeur est « Berners-Lee ».



Ressources, prédicats et objets

Les ressources

Une ressource peut être :

- un objet du Web (page, partie de page, site, document)
- un objet non accessible par le Web (livre, personne, classe)

Toujours identifiée par une URI.

Les prédicats

Une propriété décrit une ressource. Elle peut être :

- une caractéristique
- un attribut
- une relation

Sa signification définit les valeurs permises, les types de ressources décrites et ses liens avec les autres propriétés.

Les objets

La valeur d'une déclaration peut être :

- une autre ressource, désignée par une URI
- un littéral : une chaîne ou un type primitif XML

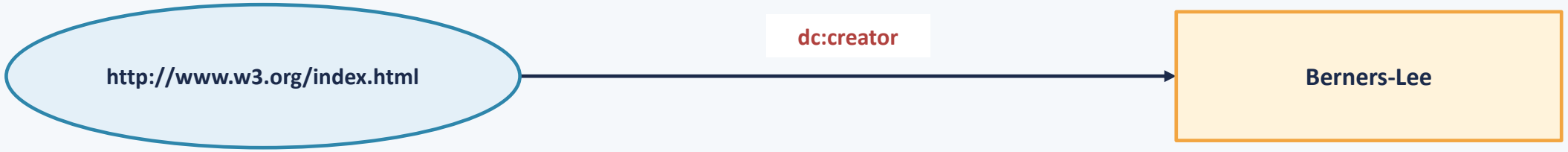
Un littéral termine le graphe : on ne peut pas continuer à partir de lui.

Le graphe RDF

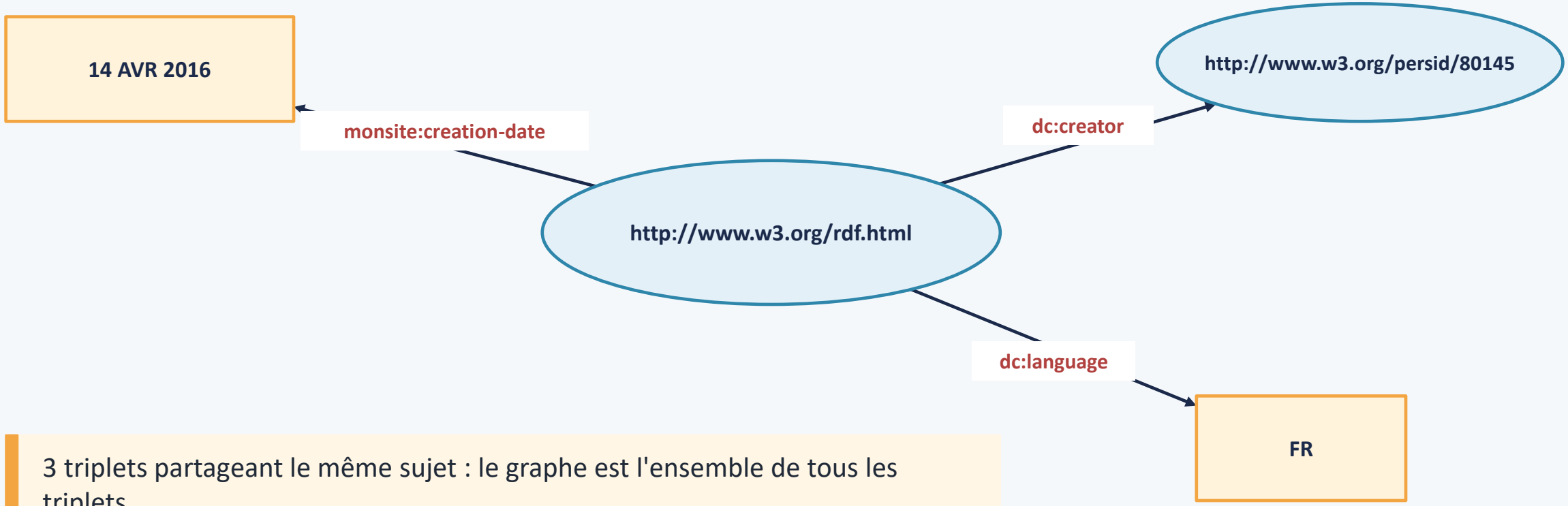
- **Nœud ovale** pour le sujet
- **Nœud ovale ou rectangle** pour l'objet (rectangle si c'est un littéral)
- **Arc orienté** du sujet vers l'objet, étiqueté par le prédicat

Un même triplet, deux écritures

Sujet (ressource)	`http://www.w3.org/index.html`
Prédicat (propriété)	`http://purl.org/dc/elements/1.1/creator`
Objet (valeur)	`http://www.w3.org/persid/80145` ou `« Berners-Lee »`



Exemple : une page décrite par plusieurs triplets



3 triplets partageant le même sujet : le graphe est l'ensemble de tous les triplets.

ressource littéral propriété

Syntaxe abstraite et syntaxes concrètes

Syntaxe abstraite

Conceptuelle, indépendante de l'implémentation, semi-formelle : elle favorise la communication.

Syntaxe concrète

Pour l'implémentation et l'échange effectif de données. Un triplet peut être codé en : Notation 3, Turtle, RDF/XML.

Triplet (sujet, prédicat, objet)



Les nœuds ovales = ressources · les arcs = propriétés (relations) · les rectangles = valeurs littérales.

RDF/XML : structure générale

- **<rdf:RDF>** : élément racine. Il indique que le document est RDF et déclare l'espace de noms RDF (et les autres espaces de noms utilisés).
- **<rdf:Description>** : contient la déclaration de plusieurs triplets relatifs à un seul sujet.

```
<?xml version="1.0"?>
<rdf:RDF
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:local="http://www.MonDomaine.dz/Vocabulary#">

  <rdf:Description rdf:about="URI-du-sujet">
    <!-- propriétés et relations du sujet -->
  </rdf:Description>

</rdf:RDF>
```

Grammaire du cours : RDF ::= [<rdf:RDF>] description [</rdf:RDF>]

Dans RDF/XML, les attributs s'écrivent rdf:about et rdf:resource (le support de cours omet parfois le préfixe rdf:).

RDF/XML : description avec une seule propriété

```
<?xml version="1.0"?>
<rdf:RDF
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:local="http://www.MonDomaine.dz/Vocabulary#">

  <rdf:Description
      rdf:about="http://www.polymtl.ca/Profs#MichelGagnon">
    <local:email>MichelGagnon@yahoo.fr</local:email>
  </rdf:Description>

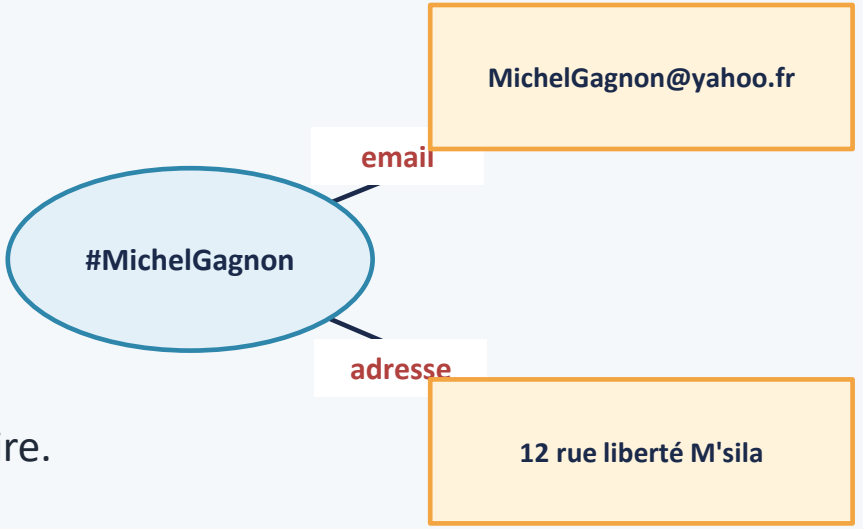
</rdf:RDF>
```



- **rdf:about** : l'URI du sujet
- **<local:email>** : le prédicat ; son contenu textuel est un littéral

RDF/XML : description avec plusieurs propriétés

```
<rdf:Description
    rdf:about="http://www.polymtl.dz/Profs#MichelGagnon">
  <local:email>MichelGagnon@yahoo.fr</local:email>
  <local:adresse>12 rue liberté M'sila</local:adresse>
</rdf:Description>
```

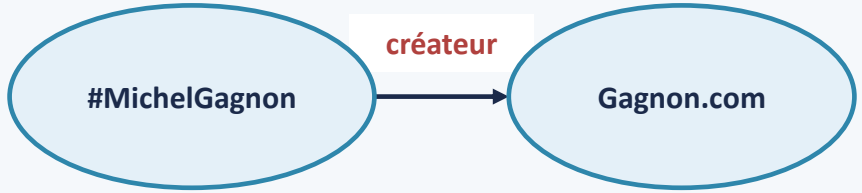


- **Une description décrit un même sujet** avec autant de propriétés que nécessaire.
- **Chaque propriété = un triplet** (même sujet, propriété différente).
- **Forme générale :** <rdf:Description rdf:about="URI"> (<propriété>valeur</propriété>)* </rdf:Description>

RDF/XML : description avec une relation

```
<rdf:Description
    rdf:about="http://www.polymtl.dz/Profs#MichelGagnon">
  <local:createur rdf:resource="http://www.Gagnon.com"/>
</rdf:Description>
```

- **Quand l'objet est une ressource** (et non un texte), on utilise l'attribut rdf:resource avec l'URI de l'objet.
- **La balise est alors vide** (<local:createur …/>) : la valeur est portée par l'attribut.

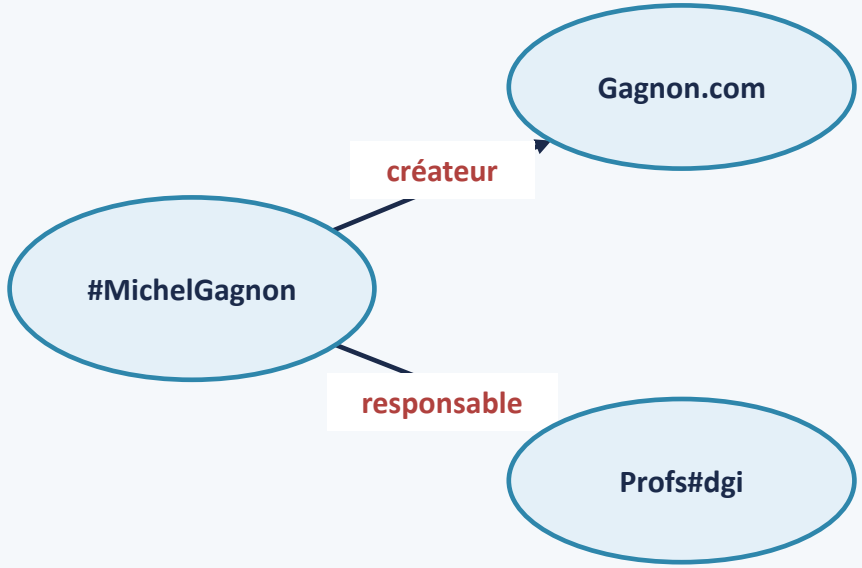


Propriété ou relation ?

Propriété = valeur littérale dans le contenu de la balise. Relation = lien vers une autre ressource via rdf:resource.

RDF/XML : description avec plusieurs relations

```
<rdf:Description
    rdf:about="http://www.polymtl.dz/Profs#MichelGagnon">
  <local:createur rdf:resource="http://www.Gagnon.com"/>
  <local:responsable
      rdf:resource="http://www.polymtl.dz/Profs#dgi"/>
</rdf:Description>
```

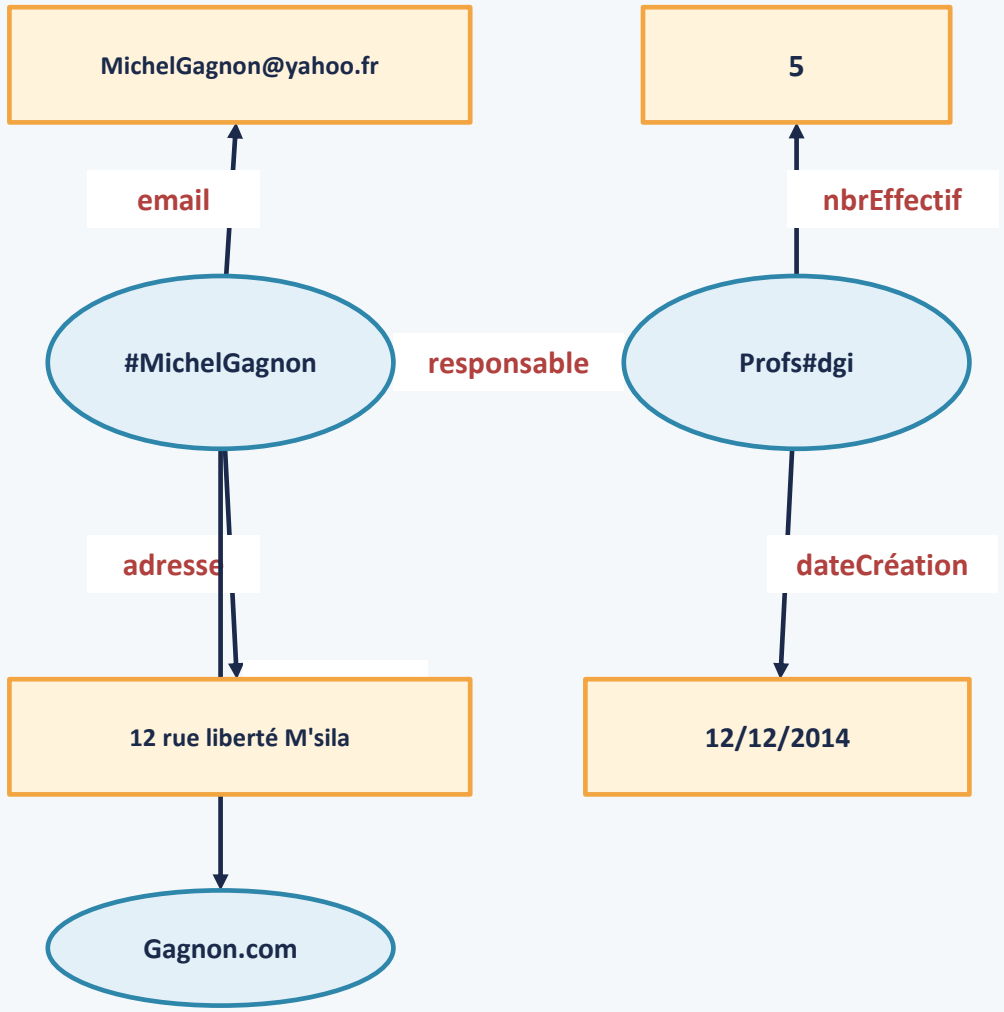


- **Forme générale :** (<relation rdf:resource="URI-objet"/>)*
- **Chaque relation = un triplet** dont l'objet est une autre ressource.
- **On peut mélanger propriétés et relations** dans la même description.

RDF/XML : cas général (propriétés + relations imbriquées)

```
<rdf:Description rdf:about="…/Profs#MichelGagnon">
  <local:email>MichelGagnon@yahoo.fr</local:email>
  <local:adresse>12 rue liberté M'sila</local:adresse>
  <local:createur rdf:resource="http://www.Gagnon.com"/>
  <local:responsable>
    <rdf:Description rdf:about="…/Profs#dgi">
      <local:nbrEffectif>5</local:nbrEffectif>
      <local:dateCreation>12/12/2014</local:dateCreation>
    </rdf:Description>
  </local:responsable>
</rdf:Description>
```

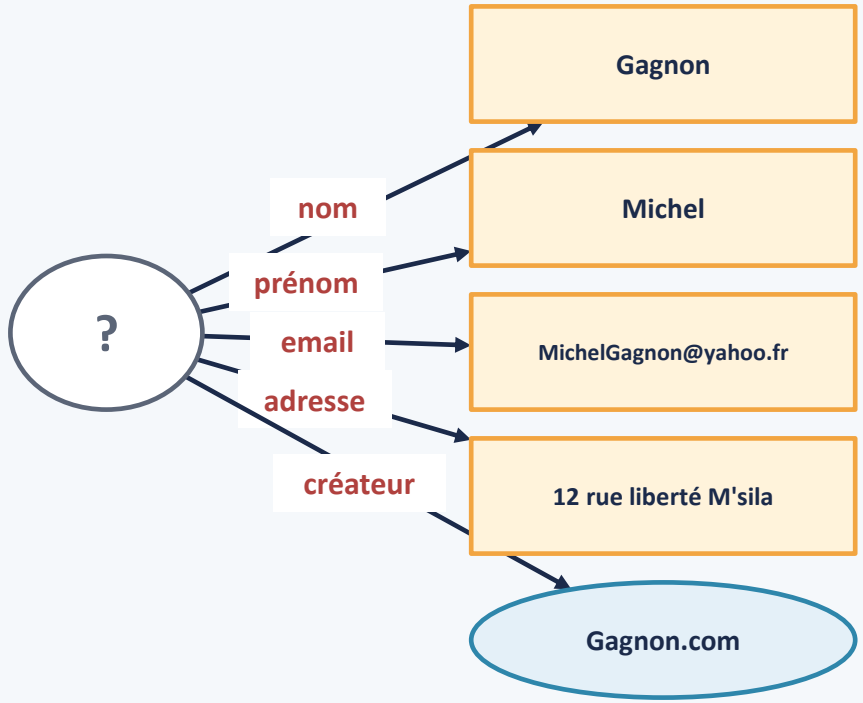
Une relation peut contenir directement une nouvelle <rdf:Description> : l'objet est décrit « à la volée ».



Ressource anonyme (nœud vide)

- Parfois on ne peut pas identifier la ressource : on ne connaît que les valeurs de ses propriétés.
- On la représente par un nœud vide (ressource anonyme) : une <rdf:Description> sans rdf:about.

```
<rdf:Description>
  <local:nom>Gagnon</local:nom>
  <local:prenom>Michel</local:prenom>
  <local:email>MichelGagnon@yahoo.fr</local:email>
  <local:adresse>12 rue liberté M'sila</local:adresse>
  <local:createur rdf:resource="http://www.Gagnon.com"/>
</rdf:Description>
```



Identificateur de nœud (rdf:nodeID)

- **Évite l'imbrication des descriptions** et rend le document RDF plus lisible.
- On donne un identifiant local au nœud, réutilisable pour ajouter d'autres triplets.

```
<rdf:Description rdf:nodeID="p304">
  <local:worksAt rdf:resource="http://www.polymtl.ca/Vocabulary#dgi"/>
  <local:hasName>Michel Gagnon</local:hasName>
</rdf:Description>

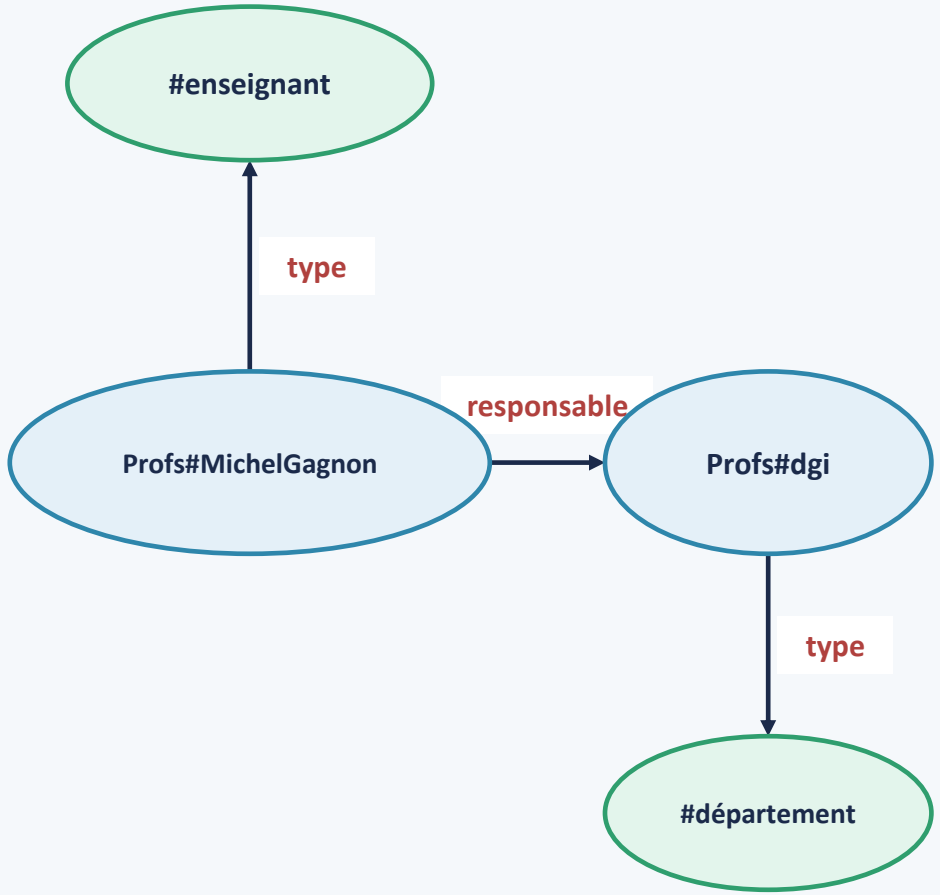
<rdf:Description rdf:nodeID="p304">
  <local:hasHomePage
      rdf:resource="http://www.professeurs.polymtl.ca/michel.gagnon"/>
</rdf:Description>
```

Les deux blocs décrivent le même nœud « p304 » : leurs triplets s'additionnent (3 triplets au total).

Typer les ressources : rdf:type

- Toutes les ressources ne sont pas de même nature : professeur, département, page personnelle…
- En RDF, on ajoute une propriété entre la ressource et son type : rdf:type (complément RDFS).

```
<rdf:Description
    rdf:about="…/Profs#MichelGagnon">
  <rdf:type rdf:resource="…/Vocabulary#Enseignant"/>
  <local:worksAt>
    <rdf:Description
        rdf:about="…/Vocabulary#dgi">
      <rdf:type
        rdf:resource="…/Vocabulary#Department"/>
    </rdf:Description>
  </local:worksAt>
</rdf:Description>
```



Les conteneurs RDF

- Un conteneur indique qu'une ressource en regroupe d'autres ; chaque élément est un membre.
- Exemples : un livre à plusieurs auteurs ; la liste des étudiants d'un groupe.
- Le conteneur est relié à ses membres par rdf:_1, rdf:_2, rdf:_3… à travers une ressource supplémentaire, typée par l'une des trois classes ci-dessous.

rdf:Bag

Groupe non ordonné. Une propriété a plusieurs valeurs sans que l'ordre ait un sens.

rdf:Seq

Groupe ordonné. L'ordre d'apparition des valeurs a un sens.

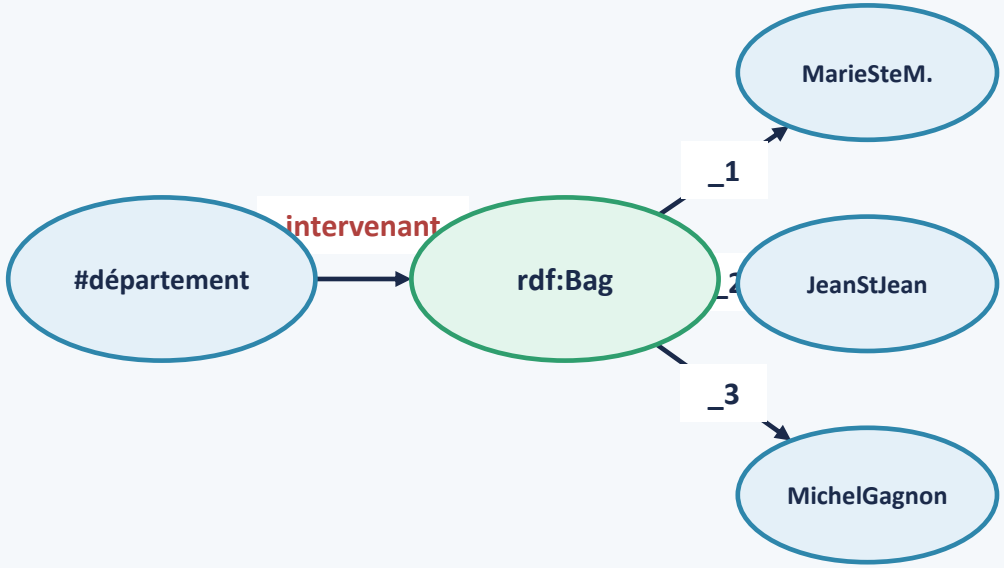
rdf:Alt

Alternatives : on s'attend à ce qu'une seule soit choisie (ex. sites de téléchargement).

rdf:Bag : groupe non ordonné

- Un groupe non ordonné de ressources ou de littéraux : plusieurs valeurs sans ordre significatif.
- Exemple : lister les enseignants d'un département.

```
<rdf:Description rdf:about="…#ProfsDepartement">
  <local:intervenant>
    <rdf:Bag>
      <rdf:li rdf:resource="…/Profs#MarieSteMarie"/>
      <rdf:li rdf:resource="…/Profs#JeanStJean"/>
      <rdf:li rdf:resource="…/Profs#MichelGagnon"/>
    </rdf:Bag>
  </local:intervenant>
</rdf:Description>
```

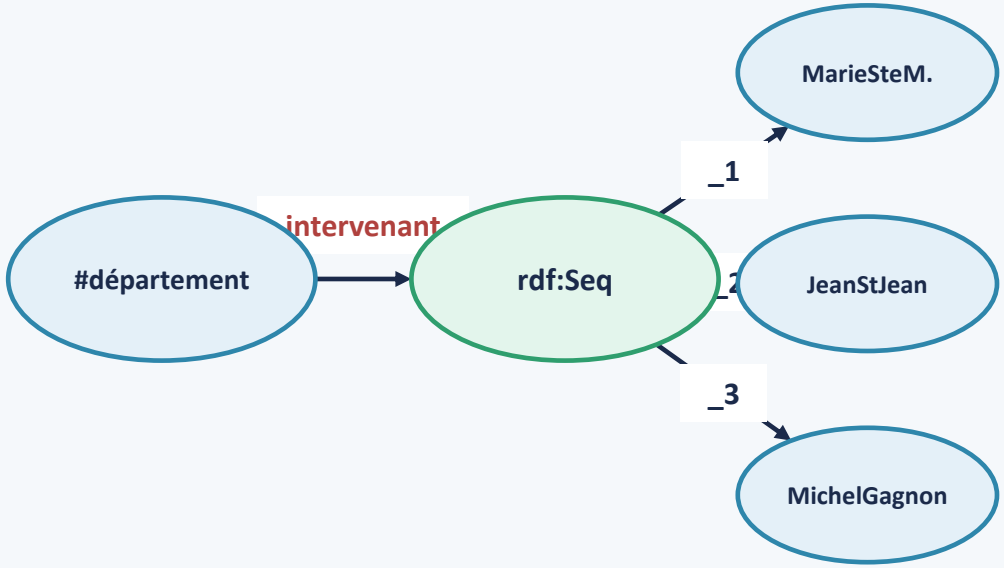


rdf:li est un raccourci de rdf:_1, rdf:_2, rdf:_3 (numérotation automatique). Le support écrit directement rdf:_1, rdf:_2…

rdf:Seq : groupe ordonné

- Un groupe ordonné de ressources ou de littéraux : l'ordre d'apparition a un sens.
- Exemple : lister les enseignants du département par ordre de grade.

```
<rdf:Description rdf:about="…#ProfsDepartement">
  <local:intervenant>
    <rdf:Seq>
      <rdf:_1 rdf:resource="…/Profs#MarieSteMarie"/>
      <rdf:_2 rdf:resource="…/Profs#JeanStJean"/>
      <rdf:_3 rdf:resource="…/Profs#MichelGagnon"/>
    </rdf:Seq>
  </local:intervenant>
</rdf:Description>
```

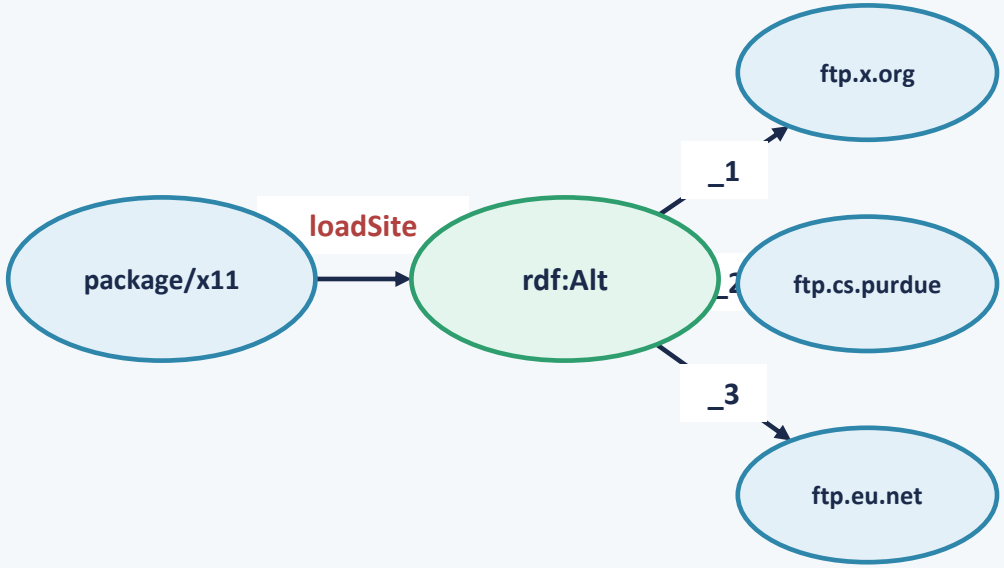


Ici _1 est le premier de la liste : changer l'ordre change le sens (le grade le plus élevé en premier, par ex.).

rdf:Alt : alternatives

- Un conteneur d'alternatives parmi lesquelles une seule est censée être sélectionnée.
- Exemple : les sites de téléchargement d'un paquet logiciel.

```
<rdf:Description rdf:about="http://www.x.org/package/x11">
  <local:loadSite>
    <rdf:Alt>
      <rdf:_1 rdf:resource="http://ftp.x.org"/>
      <rdf:_2 rdf:resource="http://www.ftp.cs.purdue.edu"/>
      <rdf:_3 rdf:resource="http://www.ftp.eu.net"/>
    </rdf:Alt>
  </local:loadSite>
</rdf:Description>
```



Le premier membre est généralement le choix par défaut.

02

RDF Schema (RDFS)

Un vocabulaire pour typer les ressources et les propriétés

Pourquoi RDFS ? Les limites de RDF

RDF permet de…

Déclarer des propriétés et des relations sur des ressources, et spécifier leur type (rdf:type).

…mais pas de décrire les types

RDF ne décrit pas les types des propriétés (relations) ni des ressources.

Questions sans réponse en RDF seul

- Quelles valeurs sont autorisées pour une propriété ?
- Quelles propriétés sont autorisées sur une ressource ?
- Quels sont les liens entre les ressources ?
- Quelle hiérarchie entre les types (spécialisation / généralisation) ?

Absence de vocabulaire
→ la solution : RDFS

RDFS : définition

- **RDFS sert à exprimer un vocabulaire**. Pour des raisons historiques on dit « RDFS » (par analogie avec XML Schema) ; on parle aussi de RDF Vocabulary Language.
- **C'est un système de typage pour RDF**, comparable à l'approche orientée objet :
  - classes et instances
  - propriétés des classes
  - hiérarchie des classes (spécialisation / généralisation)
  - mais pas de méthodes

Espace de noms RDFS

`http://www.w3.org/2000/01/rdf-schema#`

`préfixe : rdfs:`

RDF → des faits
RDFS → le vocabulaire pour les décrire

RDFS : les classes

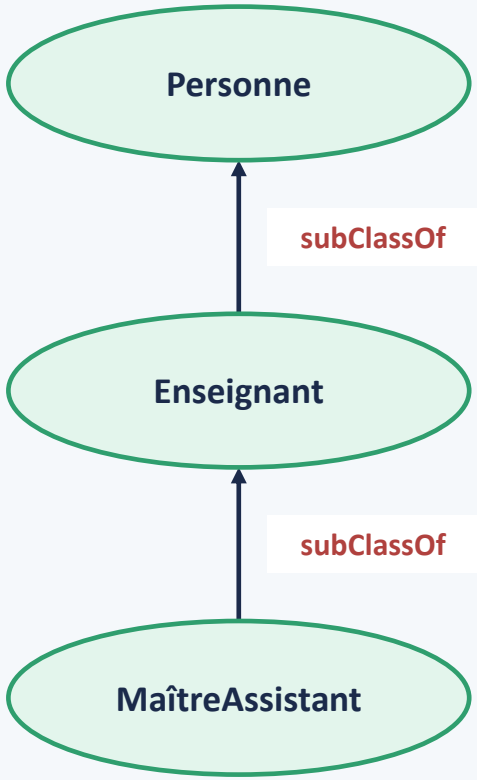
- Une classe est un type (une catégorie) qui regroupe des instances (objets, ressources) partageant les mêmes caractéristiques.
- Une classe est identifiée par une URI. Exemples : Enseignant, Étudiant, Personne.
- On la déclare avec rdfs:Class :

```
<rdfs:Class rdf:about="http://www.polymtl.ca/Vocabulary#Enseignant"/>
<rdfs:Class rdf:about="http://www.polymtl.ca/Vocabulary#Departement"/>
```



RDFS : les sous-classes (rdfs:subClassOf)

- **rdfs:subClassOf** : une classe est une sous-classe d'une autre.
- Si C est sous-classe de C’ et qu'une ressource est de type C, on déduit automatiquement qu'elle est aussi de type C’.
- La relation rdfs:subClassOf est transitive.




```
<rdfs:Class rdf:about="#MaitreAssistant">
  <rdfs:subClassOf rdf:resource="#Enseignant"/>
</rdfs:Class>
```

Transitivité : un MaîtreAssistant est aussi un Enseignant, donc une Personne.

RDFS : les propriétés, domaine et co-domaine

- Une propriété caractérise les instances d'une classe. Elle est définie par un nom, un domaine (types de ressources qui la portent) et un co-domaine / rang (valeurs autorisées).
- **rdfs:domain** définit le domaine ; rdfs:range

Nom	Domaine	Co-domaine
email	Personne	chaîne (Literal)
responsable	Enseignant	Département

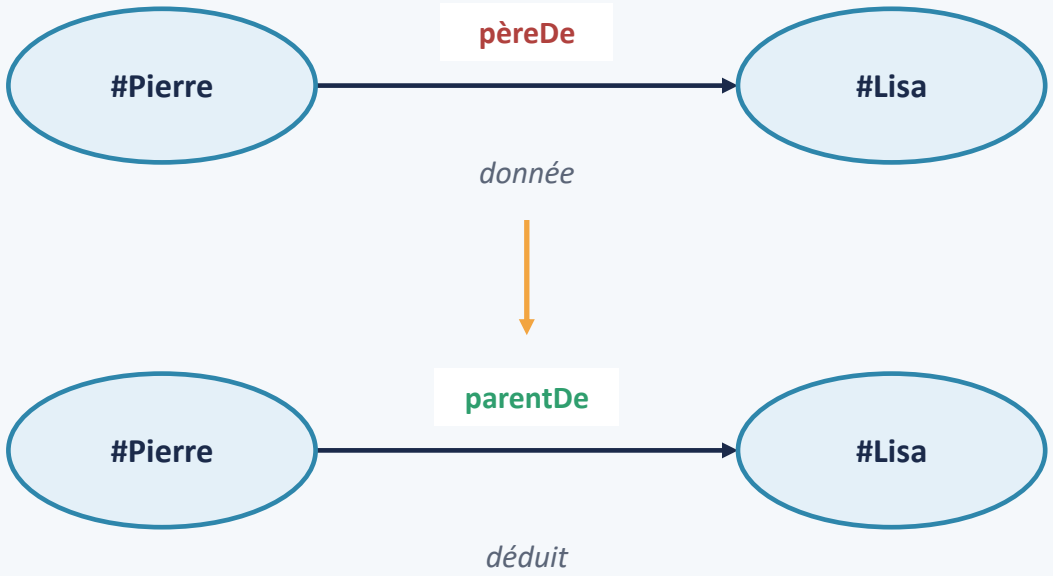
```
<rdf:Property rdf:about="#responsable">
  <rdfs:domain rdf:resource="#Enseignant"/>
  <rdfs:range  rdf:resource="#Departement"/>
</rdf:Property>
```

Domaine et co-domaine servent à déduire des types (inférence) : ils ne bloquent pas les données invalides. Pour interdire, il faut OWL.

RDFS : les sous-propriétés (rdfs:subPropertyOf)

- P’ est une sous-propriété de P : si deux ressources r1 et r2 sont liées par P’, elles le sont aussi par P.
- Exemple : père est une sous-propriété de parent.

```
<rdf:Property rdf:about="#pere">
  <rdfs:subPropertyOf rdf:resource="#parent"/>
</rdf:Property>
```



Autre exemple

chauffeur et co-chauffeur sont deux sous-propriétés possibles de conducteur.

RDFS : les littéraux

rdfs:Literal

Classe des valeurs constantes : chaînes, entiers… À utiliser comme co-domaine d'une propriété à valeur simple.

rdf:XMLLiteral

Valeur constante correspondant à un document XML bien formé.

```
<rdf:Property rdf:about="#email">
  <rdfs:domain rdf:resource="#Personne"/>
  <rdfs:range  rdf:resource="http://www.w3.org/2000/01/rdf-schema#Literal"/>
</rdf:Property>
```

Dans le support, l'exemple XMLLiteral est écrit pour #email : en pratique on choisit XMLLiteral seulement pour une valeur qui est elle-même du XML.

RDFS : documenter le vocabulaire (rdfs:comment)

- **rdfs:comment** : propriété qui permet de commenter une ressource ou une propriété.
- **Complément fréquent : rdfs:label** pour un nom lisible par un humain.

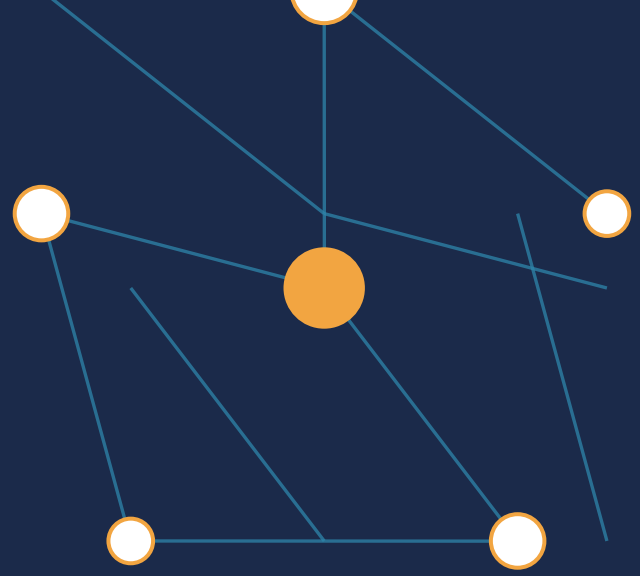
```
<rdfs:Class rdf:about="http://www.polymtl.ca/Vocabulary#Enseignant">
  <rdfs:comment>classe des enseignants</rdfs:comment>
</rdfs:Class>

<rdf:Property rdf:about="#email">
  <rdfs:comment>spécifier l'email d'une personne</rdfs:comment>
  <rdfs:domain rdf:resource="#Personne"/>
  <rdfs:range  rdf:resource="http://www.w3.org/2000/01/rdf-schema#Literal"/>
</rdf:Property>
```

Syntaxe du support : à écrire correctement

Dans le support	Forme correcte
rdfs:property	rdf:Property
rdfs:class / rdfs:Class (casse)	rdfs:Class
rdfs:subclassof / rdfs:SubProperty	rdfs:subClassOf / rdfs:subPropertyOf
rdfs:Comment / Rdf :domain	rdfs:comment / rdfs:domain, rdfs:range
about=… / rdf:ressource=…	rdf:about=… / rdf:resource=…
rdfs:XMLLiteral	rdf:XMLLiteral
Balises ouvertes mais non fermées	Chaque balise ouvrante est refermée

RDF/XML est du XML : les noms sont sensibles à la casse et le document doit être bien formé.



COMPLÉMENTS

RDF & RDF Schema

Donner un sens explicite et structuré aux ressources du Web



Les URI

URI : Uniform Resource Identifier. Pour chaque entité (ressource), il faut choisir une chaîne de texte unique, sous l'une des formes suivantes :

URL	`http://www.monsite.fr/LesMiserablesLeRoman`
URN	`urn:ietf:rfc:2141`
Tag URI	`tag:websemantique.org,2005-04-25:De_Gaulle,Charles`

Les URI — avantages et inconvénients

Avantages

- Tout le monde peut créer n'importe quelles méta-données sur n'importe quelle ressource, s'il connaît son URI.
- Il est possible d'agréger des méta-données venant d'endroits différents sur une même ressource, puisqu'elles réfèrent à la même URI.

Inconvénients

- Recherche d'un URI existant.
- Enregistrement d'un nouvel URI.

RDF (1) — le triplet

Normalisé par le W3C, RDF est un modèle conceptuel permettant de décrire des choses simplement et sans ambiguïté, à l'aide de déclarations composées d'un sujet, d'un verbe et d'un complément (ex. « Cette page a pour auteur François Martin »).



« Cette page a pour auteur François Martin »

RDF (2) — exemple

Affirmation : « Socrate est un homme » → <Socrate> <est> <un homme>

```
<?xml version="1.0" encoding="iso-8859-1"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">
  <rdf:Description about="http://grandshommes.org/Socrate">
    <rdf:type resource="http://description.org/schema/animaux/Homme"/>
  </rdf:Description>
</rdf:RDF>
```

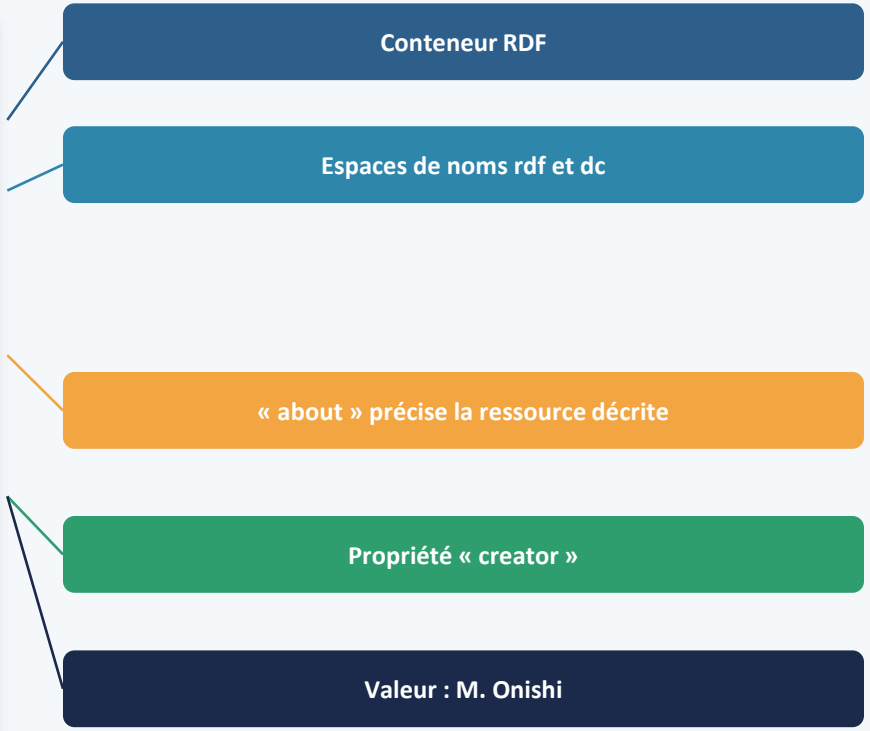
En syntaxe RDF/XML ci-dessus — il existe aussi une syntaxe abrégée équivalente, dite N3.

RDF (3) — exemple annoté (Dublin Core)

```
<?xml version="1.0"?>
<rdf:RDF
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:dc="http://purl.org/dc/elements/1.1/">

  <rdf:Description
    about="http://xml.coverpages.org/RadioTV-NewsML-en-
      20020224.pdf">

    <dc:creator>M. Onishi</dc:creator>
    <dc:title>RadioTV-NewsML in Japan</dc:title>
    <dc:date>2002-02-21</dc:date>
    <dc:type>Text</dc:type>
    <dc:format>application/pdf</dc:format>
  </rdf:Description>
</rdf:RDF>
```



Le Dublin Core est un vocabulaire (ontologie) servant à décrire des œuvres littéraires ou artistiques sur tout type de support.

RDF Schéma et OWL

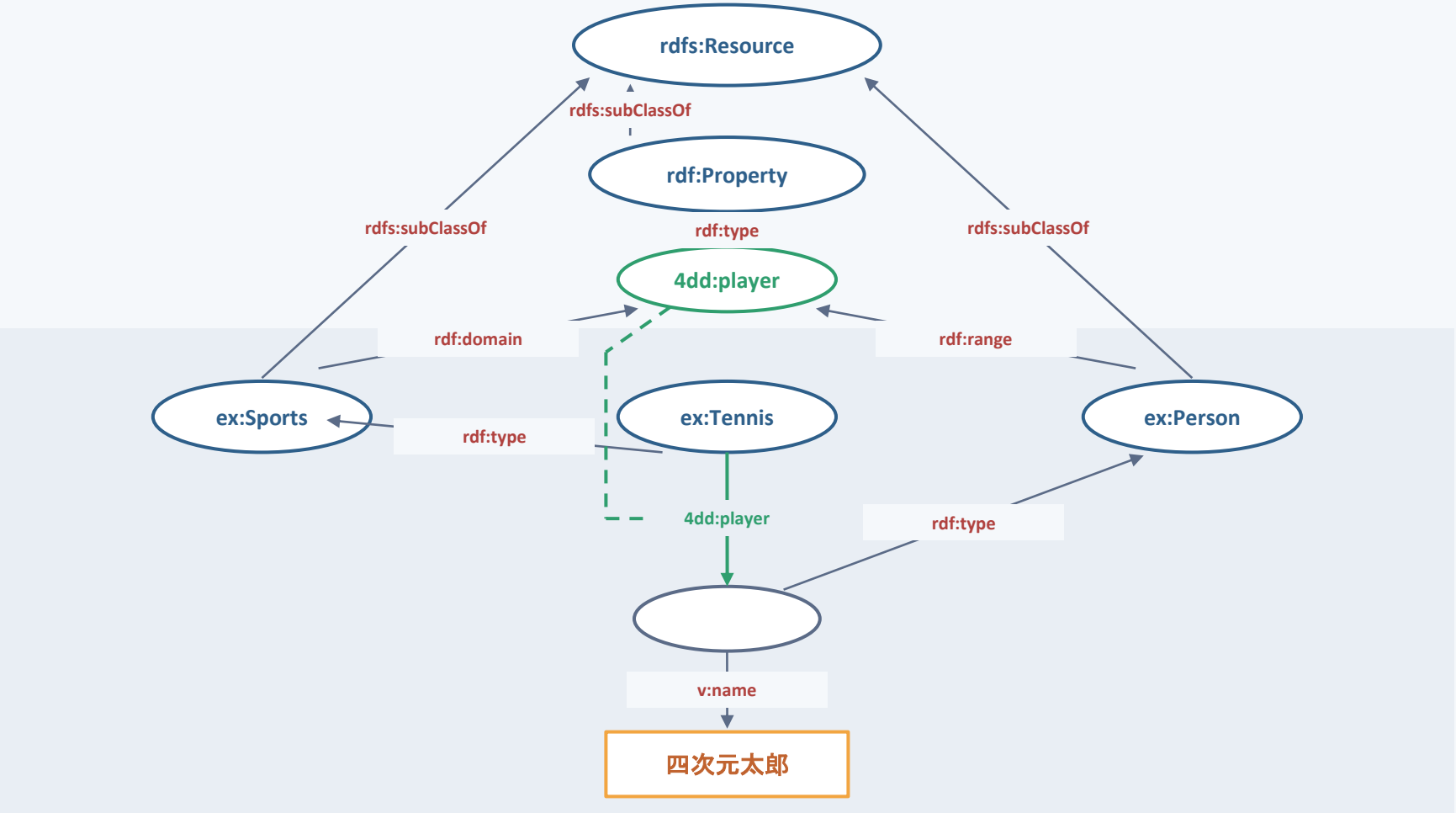
RDF Schéma permet :

- de définir un nouveau vocabulaire : des ressources avec leurs propriétés, et les valeurs possibles de ces propriétés ;

- de construire une ontologie simple (notion de sous-classe, d'instance de classe).

OWL (Web Ontology Language) est une extension de RDF Schéma, offrant une expressivité logique plus riche.

RDF Schéma — exemple de graphe



RDF et RDF Schéma

Avantages

- Rendent le contenu Web compréhensible par les machines.
- Des moteurs d'inférence peuvent établir des déductions puissantes grâce aux ontologies construites avec RDF Schéma ou OWL.
- Par l'intermédiaire des « espaces de noms », le vocabulaire utilisé n'est pas figé et peut être complété.

Inconvénients

- Nécessité d'une entente sur le vocabulaire.
- Travail fastidieux pour les créateurs de site ?

Synthèse : RDF et RDFS

RDF

Modèle de triplets (sujet, prédicat, objet) → un graphe. Syntaxes : RDF/XML, Turtle, N3. Conteneurs Bag, Seq, Alt. rdf:type pour typer les ressources.

RDFS

Vocabulaire pour décrire des ontologies légères : classes et sous-classes, propriétés et sous-propriétés, domaine et co-domaine, littéraux, commentaires.

RDFS permet de :

- Définir classes et sous-classes, et les propriétés d'une classe
- Définir les relations entre classes
- Restreindre domaine et co-domaine d'une propriété
- → décrire des ontologies légères (pour plus d'expressivité : OWL)